

Quantum Path Selection for Network Traffic Routing: a QAOA Formulation and Solution

Qubit Hackathon team

September 9, 2026

Abstract

We address the AT&T network-traffic-routing challenge with a hybrid quantum-classical pipeline: classical preprocessing enumerates a small set of feasible candidate paths per traffic demand, a quantum optimizer (QAOA on the Classiq platform) selects one path per demand by minimizing a QUBO cost Hamiltonian that trades off latency against congestion, and classical post-processing validates feasibility and computes operational KPIs. This document states the QUBO formulation, the QAOA solution method as implemented, and a worked example on a 9-qubit toy instance, including the Fortz–Thorup congestion cost [2] used both as an optional in-Hamiltonian congestion profile and as an instance-independent KPI.

1 Problem and hybrid pipeline

A communication network is a directed graph $G = (V, E)$ whose links $e \in E$ have finite capacity $c_e > 0$ and latency $\ell_e \geq 0$. Traffic consists of K demands: demand k must route bandwidth $b_k > 0$ from source s_k to target t_k , and carries a service-class priority $\pi_k > 0$ ($\pi_k = 1$ by default). The goal is to select routes that use resources efficiently while avoiding congestion and holding service quality — the challenge KPIs are congestion relief on the most loaded links, low average latency, high demand satisfaction, few route changes against the current state, and zero capacity violations.

The solution is a five-step hybrid pipeline; this document covers the combinatorial core (steps 3–4):

1. **Input** the network graph, link capacities/latencies, and the demand matrix.
2. **Classical path generation:** for each demand k , compute a small candidate set P_k of feasible paths (K -shortest paths, filtered by latency/policy constraints). The candidate sets are an *input* to the steps below.
3. **QUBO construction (Section 2):** one binary variable per candidate path; a cost Hamiltonian that selects exactly one path per demand, prefers low latency, and penalizes congestion.
4. **Quantum optimization (Section 3):** QAOA over the cost Hamiltonian, implemented on Classiq; a classical optimizer tunes the variational parameters.
5. **Classical decode and validation (Section 3.3):** map sampled bitstrings back to routes, check feasibility, and compute KPIs against a classical baseline.

Restricting each demand to a precomputed candidate set makes this a *restricted optimal general routing* (explicit-path, MPLS-style) problem in the sense of Fortz–Thorup [2], and keeps the qubit count at $N = \sum_k |P_k|$ — independent of the number of links.

2 QUBO formulation

2.1 Encoding

The decision variables are

$$x_{k,p} \in \{0, 1\}, \quad x_{k,p} = 1 \iff \text{demand } k \text{ routes on its candidate path } p \in P_k. \quad (1)$$

Variables are packed into one flat register of $N = \sum_{k=1}^K |P_k|$ qubits by

$$i(k, p) = o_k + p, \quad o_k = \sum_{j < k} |P_j|, \quad (2)$$

which supports differing candidate-set sizes $|P_k|$ per demand.

2.2 Cost Hamiltonian

The full (diagonal) cost Hamiltonian is

$$H = \lambda_{\text{1hot}} H_{\text{1hot}} + \tilde{H}_{\text{lat}} + \lambda_{\text{cong}} \tilde{H}_{\text{cong}} + \lambda_{\text{switch}} H_{\text{switch}}, \quad (3)$$

with all coefficients precomputed classically as plain floats. The same polynomial is used symbolically to build the quantum circuit and numerically to evaluate sampled bitstrings (Section 3).

One path per demand (constraint penalty).

$$H_{\text{1hot}} = \sum_{k=1}^K \left(\sum_{p \in P_k} x_{k,p} - 1 \right)^2. \quad (4)$$

$H_{\text{1hot}} = 0$ exactly on one-hot assignments; the smallest violation (zero or two selected paths for some demand) costs 1.

Latency objective. The raw cost of routing demand k on path p is priority- and bandwidth-weighted by default,

$$L_{k,p} = \pi_k b_k \sum_{e \in p} \ell_e \quad (\text{optionally unweighted: } L_{k,p} = \pi_k \sum_{e \in p} \ell_e), \quad (5)$$

so that a 10 Gbps demand on a slow path costs ten times a 1 Gbps demand, and a demand with $\pi_k > 1$ pulls proportionally harder toward its low-latency candidates when several demands contend for the same link. Latencies are normalized so that the spread of the latency term over the feasible (one-hot) space equals a single tunable scale c_{scale} :

$$\tilde{H}_{\text{lat}} = s_{\text{lat}} \sum_{k,p} L_{k,p} x_{k,p}, \quad s_{\text{lat}} = \frac{c_{\text{scale}}}{\max(L^{\text{max}} - L^{\text{min}}, \varepsilon)}, \quad (6)$$

where $L^{\text{min}} = \sum_k \min_p L_{k,p}$ and $L^{\text{max}} = \sum_k \max_p L_{k,p}$ are the exact feasible-space extremes (the latency term is separable per demand), and ε guards a degenerate zero spread.

Because the priorities enter $L_{k,p}$ before the extremes are taken, the normalization is priority-aware: the feasible-space latency spread is exactly c_{scale} for any $\{\pi_k\}$, and a global rescale $\pi_k \mapsto c \pi_k$ cancels in (6) — only the ratios between demands are meaningful. Raising a single π_k therefore reallocates a fixed latency budget toward demand k rather than inflating $\tilde{H}_{\text{lat}}$ against λ_{cong} ; the other demands' latency preferences weaken relative to congestion, which is the intended service-class semantics.

Congestion (soft capacity). The utilization of link e is linear in x ,

$$u_e(x) = \frac{1}{c_e} \sum_{(k,p): e \in p} b_k x_{k,p}, \quad (7)$$

and the congestion term averages a convex profile g over the m_{used} links that appear in at least one candidate path:

$$\tilde{H}_{\text{cong}} = \frac{c_{\text{scale}}}{m_{\text{used}}} \sum_{e \in E_{\text{used}}} g(u_e(x)), \quad g(u) = \begin{cases} u^2 & \text{quadratic (default),} \\ \alpha u + \beta u^2 & \text{fortz-thorup-fit.} \end{cases} \quad (8)$$

Priorities are deliberately absent from u_e : link utilization is a physical quantity, and the Fortz–Thorup calibration below is fitted against real loads, so weighting them by service class would distort both the congestion penalty and the reported KPIs. No slack qubits and no hard capacity constraint are used: the quadratic profile penalizes congestion smoothly and pushes solutions under capacity, while hard feasibility is verified classically at decode time (Section 3.3). Since u_e is dimensionless and $O(1)$ for sane instances, the average keeps this term $O(c_{\text{scale}})$.

The second profile calibrates g to the canonical Fortz–Thorup link cost Φ_e [2]: the piecewise-linear convex function with

$$\Phi'_e(\ell) = \begin{cases} 1 & \ell/c_e \in [0, 1/3) \\ 3 & \ell/c_e \in [1/3, 2/3) \\ 10 & \ell/c_e \in [2/3, 9/10) \\ 70 & \ell/c_e \in [9/10, 1) \\ 500 & \ell/c_e \in [1, 11/10) \\ 5000 & \ell/c_e \in [11/10, \infty) \end{cases} \implies \Phi_e(c_e) = \frac{32}{3} c_e. \quad (9)$$

A piecewise-linear function of a sum of binaries is not QUBO-expressible without auxiliary variables, so the in-Hamiltonian surrogate is the non-negative least-squares fit of $\alpha u + \beta u^2$ to Φ_e per unit capacity on $u \in [0, 1.1]$. The hockey-stick tail of Φ drives the unconstrained linear coefficient negative (which would *reward* lightly loaded links), so α is clamped at 0; the fit is a tail-calibrated pure quadratic, $\alpha = 0$, $\beta \approx 16.73$ — about $16.7\times$ the default congestion pressure. The linear part (when present) folds into the linear coefficients of H ; the quadratic part reuses the u_e^2 machinery.

Route changes (optional, default off). Given a current routing $r : k \mapsto p_{\text{cur}}(k)$ for some or all demands,

$$H_{\text{switch}} = \sum_{k \in \text{dom } r} \sum_{p \neq p_{\text{cur}}(k)} x_{k,p}, \quad (10)$$

so in the feasible subspace each rerouted demand costs exactly $\lambda_{\text{switch}} c_{\text{scale}}$. The default is $\lambda_{\text{switch}} = 0$ (the term is skipped entirely).

2.3 QUBO form

Using $x^2 = x$ for binaries, each squared affine term in (4)–(8) expands to the standard QUBO monomials; for a link e with coefficients $a_{e,i} = b_{k(i)}/c_e$ over the paths crossing it,

$$u_e^2 = \sum_i a_{e,i}^2 x_i + 2 \sum_{i < j} a_{e,i} a_{e,j} x_i x_j, \quad (11)$$

i.e. pairwise couplings between any two candidate paths sharing a link. Cross-terms between two paths of the *same* demand also appear; they are inert on the one-hot-feasible subspace (at most one of the two variables is 1) and mildly reinforce H_{1hot} outside it, so they are kept. The total Hamiltonian is therefore degree 2 — an Ising model directly implementable as a QAOA phase separator.

2.4 Weights and normalization

Parameter	Default	Role
λ_{1hot}	4.0	one-hot penalty weight
λ_{cong}	1.0	congestion weight
λ_{switch}	0.0	route-change weight (off)
c_{scale}	1.0	overall objective scale
congestion profile	quadratic	$g(u)$ in (8)
π_k (per demand)	1.0	service-class priority: latency multiplier

Table 1: Default Hamiltonian weights. All but π_k are global knobs on **QuboWeights**; the priority π_k is per-demand data carried on **Demand**.

The normalization (6) fixes the feasible-space spread of $\tilde{H}_{\text{lat}}$ to c_{scale} and keeps $\tilde{H}_{\text{cong}} = O(c_{\text{scale}})$, so the maximum feasible objective gap is $\approx 2 c_{\text{scale}}$ and $\lambda_{\text{1hot}} = 4$ dominates it: breaking the one-hot constraint is never energetically favorable. With K demands the total spread of H stays $O(10)$, keeping $e^{-i\gamma H}$ away from 2π phase wrap-around for $\gamma \in (0, \pi)$ — the normalization strategy of Classiq’s network-traffic-optimization reference notebook [3].

3 QAOA solution

3.1 Ansatz

The circuit is the standard depth- L QAOA [1] over N qubits, built on Classiq as an alternation of an arithmetic phase gadget and a transverse mixer:

$$|\psi(\gamma, \beta)\rangle = \prod_{l=1}^L \left[U_M(\beta_l) U_C(\gamma_l) \right] |+\rangle^{\otimes N}, \quad (12)$$

$$U_C(\gamma) |x\rangle = e^{-i\gamma H(x)} |x\rangle, \quad U_M(\beta) = \bigotimes_{i=1}^N R_X^{(i)}(\beta), \quad R_X(\beta) = e^{-i\beta X/2}. \quad (13)$$

U_C is generated directly from the polynomial $H(x)$ of Section 2 by Classiq’s **phase** statement (the synthesis engine compiles the degree-2 polynomial into single- and two-qubit phase rotations); sign and half-angle conventions relative to [1] are reparameterizations absorbed by the classical optimizer. The identical Python cost function is reused classically to score sampled bitstrings — one definition, two uses.

3.2 Parameter optimization

The variational objective is the shot-estimated energy

$$F(\gamma, \beta) = \langle \psi(\gamma, \beta) | H | \psi(\gamma, \beta) \rangle \approx \sum_x \frac{n_x}{S} H(x), \quad (14)$$

with n_x the observed counts over S shots. F is minimized with COBYLA, starting from the midpoint linear schedule

$$\gamma_l = \pi \frac{2l-1}{2L}, \quad \beta_l = \gamma_{L+1-l}, \quad l = 1, \dots, L \quad (15)$$

(ascending γ , descending β — an annealing-like ramp). Defaults: $L = 3$ layers, $S = 2048$ shots, at most 60 COBYLA iterations, fixed simulator seed for reproducibility. After convergence the circuit is sampled once more at the optimal parameters to obtain the solution distribution.

3.3 Decoding, feasibility, and KPIs

Each sampled bitstring is decoded classically:

- **Routes:** demand k is *satisfied* iff exactly one $x_{k,p} = 1$; the selected paths determine per-link loads (unsatisfied demands carry no traffic).
- **Feasibility:** one-hot satisfaction per demand and hard capacity checks per link ($load \leq c_e$) — the soft congestion term does not replace validation.
- **KPIs (raw units, never normalized):** per-link load and utilization, maximum utilization, capacity violations, total and bandwidth-weighted latency, demands satisfied, and the Fortz–Thorup congestion measure

$$\Phi = \sum_{e \in E} \Phi_e(\text{load}_e), \quad \Phi^* = \frac{\Phi}{\Phi_{\text{Uncap}}}, \quad \Phi_{\text{Uncap}} = \sum_k b_k \cdot \text{hopdist}(s_k, t_k), \quad (16)$$

with Φ_e the exact piecewise cost (9) (not the fit) and hopdist the unit-weight shortest-path distance. $\Phi^* = 1$ is the uncapacitated ideal; values from 1 to roughly $\frac{32}{3} \approx 10.67$ mean all traffic flows within capacity; $\Phi^* \geq \frac{32}{3}$ signals a congested network [2].

For small instances an exhaustive scan of the $\prod_k |P_k|$ one-hot assignments provides the exact optimum as a reference; QAOA quality is judged by (i) convergence of F toward that reference, (ii) the probability mass on one-hot-feasible bitstrings, and (iii) the best sampled feasible solution matching (or nearing) the reference optimum.

4 Worked example (9 qubits)

4.1 Instance

Six routers A–F, nine directed links (Table 2), three demands (Table 3). Candidate paths are the ≤ 3 latency-shortest simple paths per demand — the stand-in for pipeline step 2 — giving $|P_1| = |P_2| = |P_3| = 3$ and $N = 9$ qubits (Table 4).

The normalization constants evaluate to $L^{\min} = 66$, $L^{\max} = 102$, hence $s_{\text{lat}} = 1/36 \approx 0.0278$; all $m_{\text{used}} = 9$ links are crossed by some candidate path, so the congestion prefactor is $\lambda_{\text{cong}} c_{\text{scale}} / m_{\text{used}} = 1/9$; the scaled one-hot weight is 4. The uncapacitated Fortz–Thorup bound is $\Phi_{\text{Uncap}} = 4 \cdot 3 + 3 \cdot 2 + 3 \cdot 2 = 24$.

Link	c_e	ℓ_e
A→B	10	1
B→D	5	1
A→C	10	2
C→D	10	2
B→C	5	1
D→E	10	1
D→F	5	2
E→F	10	1
C→E	10	3

Demand	s_k	t_k	b_k	π_k
d_1	A	F	4	3
d_2	A	E	3	1
d_3	B	F	3	1

Table 3: Traffic demands (d_1 is a premium class).

Table 2: Links (capacity, latency).

Qubit	Demand	p	Path	$\sum_{e \in p} \ell_e$	$L_{k,p} = \pi_k b_k \ell(p)$
x_0	d_1	0	A–B–D–F	4	48
x_1	d_1	1	A–B–D–E–F	4	48
x_2	d_1	2	A–C–D–F	6	72
x_3	d_2	0	A–B–D–E	3	9
x_4	d_2	1	A–C–E	5	15
x_5	d_2	2	A–B–C–E	5	15
x_6	d_3	0	B–D–F	3	9
x_7	d_3	1	B–D–E–F	3	9
x_8	d_3	2	B–C–D–F	5	15

Table 4: Candidate paths and the flat index map ($o_1 = 0$, $o_2 = 3$, $o_3 = 6$).

4.2 Reference solution

Table 5 reports the exact optimum of H over all 27 one-hot assignments (the reference QAOA must reproduce), for three configurations of the congestion term. KPIs are computed by the decoder in raw units.

Congestion term	$(p_{d_1}, p_{d_2}, p_{d_3})$	$\max_e u_e$	Cap. viol.	$\sum_k b_k \ell(p_k)$	Φ^*
off ($\lambda_{\text{cong}} = 0$)	(A–B–D–F, A–B–D–E, B–D–F)	2.00	2	34	1276.1
quadratic (default)	(A–B–D–E–F, A–C–E, B–D–F)	1.40	1	40	326.3
fortz-thorup-fit	(A–B–D–E–F, A–C–E, B–C–D–F)	0.80	0	46	2.07

Table 5: Exact optimum of H under three congestion settings (all other weights at their defaults, Table 1). Latency is bandwidth-weighted; Φ^* uses the exact piecewise link cost.

The example is built so the pure-latency optimum overloads shared links: all three demands prefer the cheap corridor through B→D (capacity 5), which with $\lambda_{\text{cong}} = 0$ carries load 10 ($u = 2.0$) alongside a second violation on D→F. Switching on the default quadratic term changes the optimum — the congestion term demonstrably reshapes the routing — but at $\lambda_{\text{cong}} = 1$ the latency term still dominates and B→D stays overloaded. The Fortz–Thorup-calibrated profile ($\beta \approx 16.73$) prices overload realistically and drives the optimum to a fully feasible routing: zero capacity violations, maximum utilization 0.80, and $\Phi^* = 2.07$ (well below the congestion threshold $32/3$), bought with

+35% bandwidth-weighted latency ($34 \rightarrow 46$). This is exactly the congestion-relief-versus-latency trade-off of the challenge KPIs, exposed through the λ_{cong} / profile knobs.

Effect of the priority. The contention that π_k resolves is between d_1 and d_3 : both want B→D, whose capacity 5 cannot carry $b_1 + b_3 = 7$, so exactly one of them takes a low-latency route and the other detours. At neutral priorities ($\pi_k \equiv 1$) the feasible optimum routes d_3 over B–D–E–F and pushes the premium demand onto A–C–D–F ($\ell = 6$), for a bandwidth-weighted latency of 48 and $\Phi^* = 2.08$. Raising π_1 to 2 flips the decision, and it stays flipped for every larger value tested (up to $\pi_1 = 8$), so the demo does not sit on a knife edge: d_1 takes A–B–D–E–F ($\ell = 4$) while d_3 detours over B–C–D–F ($\ell = 5$). Both routings are capacity-clean, so this is a genuine preference change rather than a feasibility artifact — Φ^* moves only from 2.08 to 2.07 and the maximum utilization is 0.80 either way, while the premium demand’s own latency drops by a third.

4.3 QAOA run

The demo notebook (`qaoa_routing.ipynb`) synthesizes the 9-qubit, $L = 3$ ansatz (12) for this instance, optimizes (γ, β) with COBYLA ($S = 2048$ shots, ≤ 60 iterations) on the Classiq executor, and reports: the convergence of F against the brute-force optimum of Table 5 as a reference line, the probability mass on one-hot-feasible bitstrings, the top-10 sampled solutions with full KPI rows, and the best feasible sample versus the exact optimum. Success is judged by the soft criteria of Section 3.3 (COBYLA on shot noise is not asserted to hit the exact optimum).

5 Scaling and outlook

Qubit economy. The path-based encoding uses $N = \sum_k |P_k| \approx K\bar{k}$ qubits for K demands with $\bar{k}$ candidate paths each, independent of network size; an edge-based flow-conservation encoding of the same 6-node instance [3] needs $K \cdot |E| = 27$ qubits versus our 9, and the gap widens with the network. The price is the classical restriction to candidate sets — path quality is bounded by step 2 of the pipeline.

Knobs. λ_{cong} and the congestion profile trade latency against congestion relief (Table 5); the per-demand priorities π_k arbitrate between demands competing for the same scarce link, without touching the physical congestion model; λ_{switch} with a current routing penalizes unnecessary route changes; c_{scale} , layer count L , shots, and iteration budget control the QAOA itself.

Next steps (pipeline step 5). Compare QAOA-selected routings against a classical baseline (MILP over the same candidate sets, and the Fortz–Thorup LP bound for unrestricted routing) on the full KPI set: most-congested-link relief, average latency, demand satisfaction, route changes, and zero capacity/resiliency violations.

References

- [1] E. Farhi, J. Goldstone, and S. Gutmann, *A Quantum Approximate Optimization Algorithm*, arXiv:1411.4028 (2014).
- [2] B. Fortz and M. Thorup, *Internet Traffic Engineering by Optimizing OSPF Weights*, Proc. IEEE INFOCOM 2000, pp. 519–528.

- [3] Classiq Technologies, *Network Traffic Optimization* (QAOA application notebook), https://github.com/Classiq/classiq-library/tree/main/applications/telecom/network_traffic_optimization.